

Are you using LDAP over SSL/TLS? – Cloud OS

 kurtroggen.wordpress.com/2018/08/03/are-you-using-ldap-over-ssl-tls

Kurt Roggen [BE]

August 3, 2018

23 Votes

Today, many applications and devices connect to Active Directory over LDAP. Many of those are still performing unsecure LDAP “simple binds” where credentials are transferred in clear text over the network. Those exposed credentials typically include the “service account” used to connect to LDAP, but also include the user credentials used during the application login.

Also note that the terms “LDAP over SSL” and “LDAP over TLS” are used interchangeably. By default, LDAP communications between client and server applications are not encrypted. This is especially problematic when an LDAP simple bind is used.

LDAP Binds

The function of the “LDAP Bind” operation is to allow authentication information to be exchanged between the LDAP client and LDAP server. This LDAP authentication process supports 3 approaches:

1. Simple bind
2. SASL (Simple Authentication and Security Layer) bind
3. Sicity bind

Simple Bind

With a LDAP Simple Bind, the credentials (user name and password) used to bind the LDAP client to the LDAP server are passed over the network unencrypted.

This approach is the most “simple” but also most unsecure. Alternatives, like SASL and LDAPS should be considered.

Simple Authentication and Security Layer (SASL)

Active Directory supports the optional use of integrity verification or encryption that is negotiated as part of the SASL [authentication](#) and it also referred to as **sign** (verification/integrity) and **seal** (encryption).

Most Microsoft MMC snap-ins use sign and seal.

Simple Authentication and Security Layer (SASL) is a method for adding authentication support to connection-based protocols like LDAP and supports several authentication methods, like GSS_SPNEGO, GSSAPI, EXTERNAL, DIGEST-MD5 as described [here](#). GSS_SPNEGO will result in using Kerberos or NTLM as the underlying authentication protocol. GSSAPI always uses Kerberos as underlying authentication protocol.

Sicily

Active Directory also supports this authentication approach during LDAP binds and is intended for compatibility with legacy systems and will result in NTLM being used as underlying authentication protocol.

LDAP over TLS (aka LDAPS)

A mechanism that uses TLS to secure communication between LDAP clients and LDAP servers to avoid unsecure simple bind or clients not supporting SASL. Active Directory does not require, but supports, the use of an SSL/TLS-encrypted connection when performing a simple bind. There are 2 approaches possible:

1. LDAPS over port **636** (DC) or port **3269** (GC) where the connection is considered to be immediately secured by the certificate. SSL/TLS is negotiated before any LDAP traffic is exchanged.
2. LDAP using StartTLS over port **389** (DC) or **3268** (GC) where the StartTLS operation is used to establish secure communications. It requires the LDAP client to support this StartTLS operation.

Anyhow, both approaches require a valid certificate to establish a secure connection.

NOTE: Active Directory exposes different ports for querying the domain (controller) partition over **389** (DC/StartTLS) **or 636** (DC/LDAPS) and for querying the cross-domain partitions using the global catalog (GC) port **3268** (GC/StartTLS) **or 3269** (GC/LDAPS).

How to solve this in 3 steps?

Step 1. Detection of applications using unsecure LDAP communications – [How?](#)

Step 2. Remediation

- Part #1 – Provide secure LDAP over SSL/TLS communications over port TCP/636 for applications – [How?](#)
- Part #2 – Prevent unsecure LDAP communications over port TCP/389 (DC) and TCP/3268 (GC) by requiring LDAP Signing at the domain controller – [How?](#)

Step 3. Validation

Step 1. Detection of unsecure LDAP communications

To be able to remediate these “problem” applications, you need to inventorize which applications and/or devices are performing unsecure LDAP communications. The objective is to move all these applications/devices to LDAPS over TCP/636.

This can be achieved by looking through the events in the “Directory Service” eventlog of the domain controllers with eventid **2889**. These events include the source IP addresses, connecting user account and LDAP binding type.

To retrieve all IP addresses from these events, use my PowerShell function **Get-ADLDAPUnsecureConnection** from my [GitHub repository](#).

NOTE: It is required to increase the “LDAP Interface Events” diagnostic level on the domain controller to report when insecure binds occur.

```
# Enable LDAP logging
reg add HKLM\SYSTEM\CurrentControlSet\Services\NTDS\Diagnostics /v "16 LDAP Interface Events" /t REG_DWORD /d 2
```

or use my PowerShell functions **Get-ADDomainControllerDiagnostics/Set-ADDomainControllerDiagnostics** from my [GitHub repository](#).

```
PS:\> Set-ADDomainControllerDiagnostics -LDAP Enabled -ComputerName DC01
```

For more detailed information: TechNet blog post "[Identifying Clear Text LDAP binds to your DC's](#)"

Step 2. Remediation

The remediation process is two folded:

- Part #1: Provide secure LDAP over SSL/TLS
- Part #2: Prevent unsecure LDAP communications over TCP/389 and TCP/3268

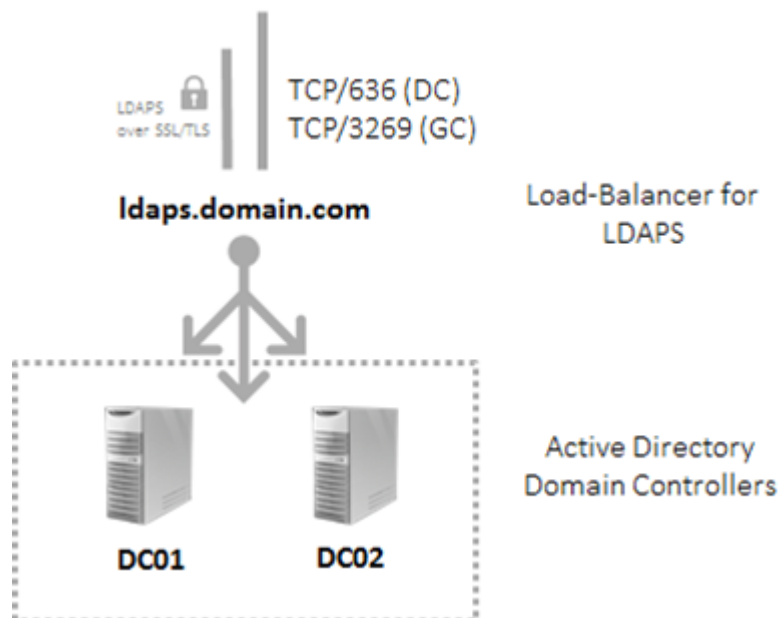
Let's have a look...

Part #1 – Provide secure LDAP over SSL/TLS communication

To make LDAP communications secure, use Secure Sockets Layer (SSL) or Transport Layer Security (TLS) by installing a valid certificate from either an internal or external/public certification authority.

Some applications will want/need to validate the LDAPS server certificate (including signing CA certificate) as part of the connection process to Active Directory. Using a single, common LDAPS certificate on all domain controllers simplifies the configuration and reduces administrative efforts.

Some applications will allow only a single LDAP server for authentication to be specified in the application. Using a load-balancer in front of the domain controllers ensures high-availability and fault tolerance for both TCP/636 (DC using LDAPS) and TCP/3269 (GC using LDAPS) communication.



Certificate Installation Guidance:

Be sure to drop your LDAPS certificate in the **personal** (My) certificate store of the **service** “**Active Directory Domain Services**” and **leave** the default certificate in the personal (My) certificate store of the local machine alone (used for Active Directory Web Services). The **ADDS (service) personal certificate store** is preferred/selected above the personal certificate store of the local machine. This way, a wildcard certificate or a SAN-based certificate (including “ldaps.domain.com” in the Subject Alternative Names) can be used for LDAPS-only purposes.

For more information about the certificate requirements for LDAPS, have a look at [KB321051](#).

IMPORTANT: When importing a relevant certificate for LDAPS, it will be used immediately without need to restart the ADDS service or ADDS domain controller.

To find out which certificate is bound to your domain controller for LDAPS, see my PowerShell function **Get-ADDomainControllerCertificate** from my [GitHub repository](#) with instructions below.

Part #2 – Prevent unsecure LDAP communication: Require LDAP Signing

The security of the directory server can be significantly improved by configuring the server

- to reject **Simple Authentication and Security Layer (SASL) LDAP binds** that do not request signing (integrity verification)
- to reject **LDAP simple binds** that are performed on a clear text (**non-SSL/TLS-encrypted**) connection

When requiring LDAP signing on domain controllers, the LDAP data-signing option must be negotiated during LDAP communications, unless Transport Layer Security/Secure Sockets Layer (TLS/SSL) is used. So, this setting does **not** have any impact on LDAP simple bind through SSL (LDAP TCP/636).

If signing is required, then LDAP simple binds not using SSL are rejected (LDAP TCP/389), hence preventing unsecure LDAP communications.

Requiring LDAP signing will also protect against replay attacks and man-in-the-middle (MITM) attacks.

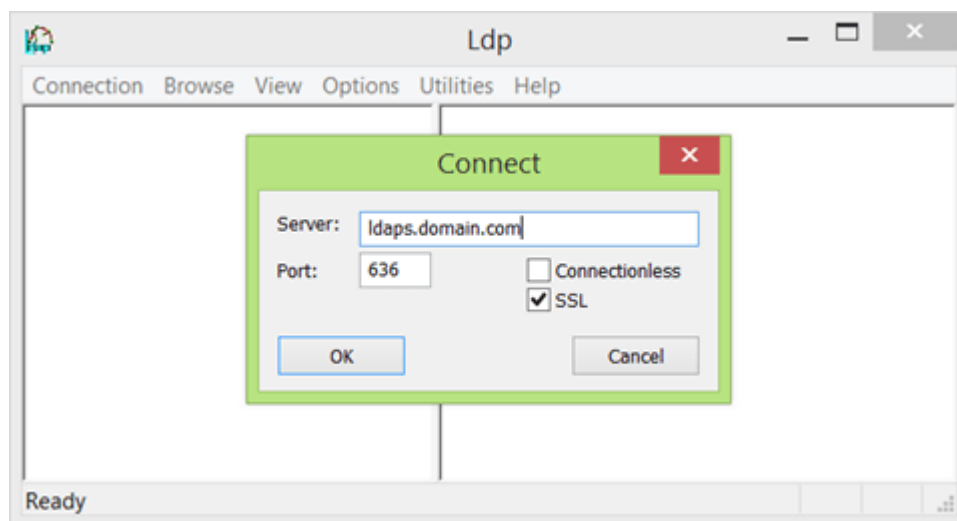
For more information about enabling the LDAP Signing requirement, have a look at [KB935834](#).

Step 3. Validation

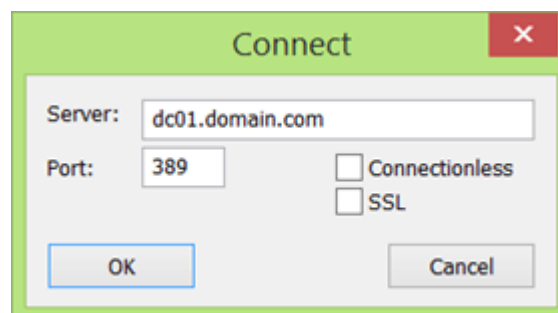
Finally, validation is in order to make sure that all behaves as expected, meaning:

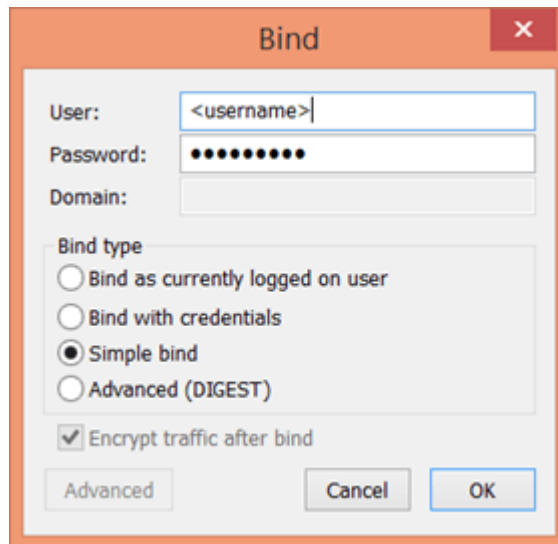
- LDAPS is working as expected
- unsecure LDAP is prevented

Part #1 To confirm that LDAP over SSL/TLS is working correctly, use ldp.exe (installed as part of RSAT Active Directory Domain Services Tools). Connect to the common LDAPS FQDN (ldaps.domain.com) over LDAPS (TCP/636).



Part #2 To confirm that unsecure LDAP simple binds no longer work, use ldp.exe again to perform a “simple bind” to a domain controller.





IMPORTANT: A “simple bind” over LDAPS (TCP/636) will continue to work, but poses no security risk as all LDAPS communication is now encrypted using SSL/TLS. However, a simple bind over LDAP (TCP/389 or TCP/3268) should no longer work.

Part #3 To confirm that all domain controllers are using the same certificate for LDAPS, use my PowerShell function **Get-ADDomainControllerCertificate** from my [GitHub repository](#).

```
PS:\> Get-ADDomainControllerCertificate -Domain "domain.com" | Select
ComputerName,Port,Subject,Thumbprint
```

ComputerName	Port	Subject	Thumbprint
DC01.domain.com	636	CN=ldaps.domain.com	23B9AFD0FOPBEE3FC3984210DFF31E801B69FFE8
DC02.domain.com	636	CN=ldaps.domain.com	23B9AFD0FOPBEE3FC3984210DFF31E801B69FFE8

Hope this helps...

Kurt Roggen [BE]

Related reading:

- Technet Blogs – [Identifying Clear Text LDAP binds to your DC's](#)
- Technet Blogs – [Creating Custom Secure LDAP Certificates for Domain Controllers with Auto Renewal](#)
- Technet Blogs – [Speaking in Ciphers and other Enigmatic tongues...update!](#)
- Technet Blogs – [Understanding LDAP Security Processing](#)
- Technet Blogs – [Troubleshooting LDAP Over SSL](#)
- [KB321051](#) – How to enable LDAP over SSL with a third-party certification authority
- [KB935834](#) – How to enable LDAP signing in Windows Server 2008

- [KB245030](#) – How to restrict the use of certain cryptographic algorithms and protocols in Schannel.dll
- LDAP v3 [RFC2251](#)